Plan 08 v0: Learned Memory Wrapper for Long-Context RCA

Progress / Fact Record

Mingjia Shi

RCA Foundation Model / Long-Context Memory

June 4, 2026

Combined progress / fact record

Weekly inputs are appended below in chronological order.

Reporting Period: May 2026 Week 4

Date range: Mon 2026-05-25 to Sun 2026-05-31

Month/week is determined by the Monday of the reporting week.

Background

- RCA and debugging tasks are becoming long-context problems: logs, traces, stack traces, test failures, code context, topology, and prior observations often exceed a practical prompt budget.
- Direct long-context prompting is expensive and brittle: high prefill cost, large KV cache, lost-in-the-middle behavior, and no persistent learning between chunks.
- Directly fine-tuning an 8B base on our small RCA/debug data is risky because we do not have the original pretraining mix needed to protect the base model's general capability.
- This project starts from a frozen 7B/8B open-source base and trains a separate wrapper memory module.

Significance

- **Research value**: formulate long-context compression as next-state prediction over learned memory states:

$$m_t = G_\phi(m_{t-1}, c_t), \qquad y_t = F_\theta(q_t, m_t)$$

- **Product value**: build an RCA foundation model for structured root-cause analysis, with lower context cost and better evidence retention.
- **Risk control**: train only the wrapper in v0, keeping the base model frozen to reduce catastrophic forgetting.
- **Community impact**: release a model, benchmark protocol, demo, and technical report rather than only internal experiment logs.

Progress / Fact Record

- **Done**: scoped Plan 08 v0 as a learned-memory wrapper project rather than full self-modifying model training.
- **Done**: read the `rca-demo` SFT results as project initialization: SFT can teach RCA signals, but it is not always a win and is not enough for robust RCA foundation-model capability.
- **Done**: defined two deliverable tracks:
  1. Research paper: new architecture for memory-state next prediction.
  2. RCA foundation model: public RCA datasets first, with public debug traces as a later extension.
- **Done**: added English and Chinese notes plus a dedicated v0 budget.
- **Done**: created the progress/fact-record slide structure.
- **Not verified yet**: no wrapper training result, LoCoMo result, public debug-trace result, or compression demo metric has landed yet.

Status Legend

- **Done**: implemented, documented, or measured from existing artifacts.
- **Verified**: backed by a completed run, metric, or reproducible output.
- **Planned**: design direction or future milestone, not necessarily immediate.
- **TODO**: near-term action to execute next; should be actionable this week / next week.
- This deck separates project facts from plans so readers do not confuse proposals with results.

Status Summary

Done

- Plan 08 v0 scoped.
- RCA-demo SFT boundary read.
- Research/project tracks defined.
- EN/ZH notes and v0 budget written.
- Slide structure created.
- Demo scripts identified.

Planned

- Frozen 8B base + wrapper path.
- LoCoMo / LongBench scaling.
- RCA foundation-model path.
- Trainable wrapper memory.
- Research paper after validation.

TODO

- Select first demo case.
- Generate four prompt variants.
- Run $q25_{base}$ / $q25_{mix}$.
- Score demo outputs.
- Package comparison table.

RCA-Demo Initialization: SFT Boundary Search

- Goal: use `rca-demo` to find how far standard SFT can push a general model toward RCA behavior.
- Setup:
  - Displayed base family: Qwen2.5-7B-Instruct (q25).
  - 4 RCA datasets: Nezha, OpenRCA-500, RCAEval, lincyaw/rca.
  - Training schedules: base, curriculum chain, and joint mix.
  - Evaluation matrix: RCA metrics + general capability benchmarks.
- Project role: this is not the final solution; it is the initialization phase that maps the SFT capability boundary.

SFT Wins: The Model Can Learn RCA Signals

- $q25_{mix}$ became the strongest complete row by `recommendation_f1`:
  - OpenRCA-500: 0.305
  - RCAEval: 0.733
  - lincyaw: 0.242
- After base-mismatch re-eval, $q25_{curr\_4\_lincyaw}$ showed real cumulative curriculum signal:
  - lincyaw `service_set_f1`: 0.272 pre-fix → 0.563 post-fix.
  - lincyaw `primary_service_match`: 0.454 → 0.723.
  - RCAEval `severity_match`: 0.041 → 0.818.
- Takeaway: SFT is useful for injecting RCA structure, service identification, and recommendation behavior.

SFT Boundaries: Not Always a Win

- General benchmark regression appears after direct RCA SFT:
  - $q25_{mix}$ GSM8K drops $0.795 \to 0.715$.
  - $q25_{mix}$ TruthfulQA drops $0.647 \to 0.589$.
- Takeaway: SFT can teach RCA behavior, but general-capability regression must be tracked before scaling.

Why Wrapper Memory Follows From RCA-Demo

- Direct base-model SFT exposes three project risks:
  1. capability drift without the original pretraining distribution;
  2. output-format fragility under LoRA updates;
  3. evidence-grounding loss even when JSON surface form improves.
- Frozen base + wrapper separates concerns:
  - base model keeps general reasoning and coding capability;
  - wrapper absorbs long context, tool observations, and RCA/debug traces;
  - failures can be isolated to memory compression instead of the whole model.
- This turns `rca-demo` from a final SFT path into evidence for the RCA foundation-model architecture choice.

First Delivery: A Demo, Not the Full System

- **Planned**: first delivery is a small, convincing demo, not a full paper-quality model release.
- The demo is intended to test the direction:
  1. long context is expensive / brittle;
  2. simple SFT has a capability boundary;
  3. frozen base + wrapper memory is a plausible next step;
  4. the same incident can be compared across full context, summary, retrieval, and memory.
- **TODO**: produce one reproducible case, one comparison table, and one qualitative walkthrough as the next delivery.

Demo Segment 1: Problem Setup

- **TODO**: pick one RCA/debug case with enough evidence to be genuinely long-context.
- Show the raw context shape:
  - issue or incident description;
  - logs / traces / failing test output;
  - code or service context;
  - distractor evidence.
- **Not verified yet**: we have not selected the demo case or measured its token/KV footprint.
- Management question: can the model identify root cause without reading everything directly?

Demo Segment 2: Baseline Comparison

- **TODO**: run the frozen 8B base with three non-learning strategies:
  1. full context;
  2. fixed-budget summary;
  3. retrieval top-k chunks.
- **TODO**: measure:
  - answer quality and evidence grounding;
  - prompt tokens / estimated KV cost;
  - format stability.
- **Not verified yet**: no full-context / summary / retrieval result has been run for the demo case.

Demo Segment 3: Wrapper Memory Path

- **Planned**: process the same context as a chunk stream:

$$m_t = G_\phi(m_{t-1}, c_t)$$

- **Planned**: generate the RCA/debug answer from compressed memory:

$$y_t = F_\theta(q_t, m_t)$$

- **TODO**: use deterministic memory compression for the immediate demo path.
- **Planned**: replace deterministic memory with trained explicit memory tokens after the demo.
- **Not verified yet**: no trained wrapper output has been measured.
- Constraint: keep the base model frozen; all adaptation happens in the wrapper.

Demo Segment 4: What We Need to Show

- The demo does not need to beat every benchmark yet.
- **Target, not verified**: it needs to show a credible direction:
  - lower prompt/KV cost than full context;
  - better evidence retention than naive summary;
  - more complete causal reasoning than retrieval-only;
  - no base-model finetuning required.
- **Decision gate**: only scale to LoCoMo / LongBench / public debug traces after the demo beats cheap baselines on at least one meaningful axis.

Demo Segment 5: Deliverable Package

- **TODO**: demo script that builds one long-context case and produces four prompts.
- **TODO**: output table comparing full context, summary, retrieval, and wrapper memory.
- **TODO**: qualitative page showing where each strategy succeeds or fails.
- **TODO**: short technical note explaining whether the result motivates wrapper training.
- **Decision**: continue to wrapper training only if the demo beats the cheap baselines on at least one meaningful axis.

Demo Assets Already Available in RCA-Demo

- **Done**: `rca-chat` Gradio probe exists and can load matrix checkpoints:
  - $q25_{base}$, $q25_{mix}$, $q25_{curr\_*}$;
  - public RCA datasets plus optional private Liang DTS bundle.
- **Done**: long-context builder script exists:
  - `scripts/compression/build_long_context_rca_benchmark.py`;
  - converts prepared `cases.jsonl` into chunked RCA items with distractors.
- **Done**: prompt-strategy preparation script exists:
  - `scripts/compression/prepare_compression_eval.py`;
  - emits `full_context`, `summary`, `retrieval`, and `learned_memory` prompt rows.
- **Not verified yet**: no end-to-end generation/score run has been completed for this demo path.

Demo Case Selection

- **Preferred public demo**: one RCAEval case.
  - It has microservice fault-injection structure.
  - It supports service/fault-type metrics.
  - It is public and suitable for release.
- **Alternative public demo**: one OpenRCA-500 case.
  - Wider service vocabulary.
  - Good for showing service-level root-cause ranking.
- **Internal-only qualitative demo**: Liang / Nokia DTS.
  - Useful for realism.
  - Not releasable; do not include raw prompts or per-case outputs in public slides.

Demo Build Commands

Step 1: build long-context RCA items

```
python3 scripts/compression/build_long_context_rca_benchmark.py \
  --cases runs/rcaeval_v1/prepared/cases.jsonl \
  --out runs/long_context_rca_demo/items.jsonl \
  --chunk-chars 1800 \
  --overlap-chars 150 \
  --distractors 2
```

Step 2: prepare four prompt strategies

```
python3 scripts/compression/prepare_compression_eval.py \
  --input runs/long_context_rca_demo/items.jsonl \
  --out runs/long_context_rca_demo/prompts.jsonl \
  --summary-chunks 8 \
  --retrieval-k 8 \
  --memory-records 8
```

Demo Four-Way Comparison

Strategy	What it uses	What we learn
Full context	All chunks in prompt	Upper-bound quality, high token/KV cost.
Summary	Fixed-budget compressed text	Cheap baseline; may lose evidence details.
Retrieval	Top-k salient chunks	Good for evidence lookup; may miss causal synthesis.
Learned memory	Rendered wrapper memory state	Whether memory can retain evidence at lower context cost.

Not verified yet: the current `learned_memory` row is a deterministic memory-compressor baseline until the trainable wrapper is run.

Demo Output Layout and Metrics

- **Planned output layout** mirrors matrix cells:
  - `runs/long_context_rca_demo/eval/<model_id>/<strategy>/predictions.jsonl`
  - `metrics.json`
  - `case_scores.jsonl`
  - `eval_command.txt`
- **Metrics to report**:
  - RCA quality: `recommendation_f1`, `root_cause_f1`, `primary_service_match`, `service_set_f1`;
  - grounding: `evidence_overlap`;
  - format: `json_parseable`, `json_repaired`, `degenerate`;
  - compression: prompt token estimate, compression ratio, latency if available.
- **TODO**: wire `prompts.jsonl` into generation and scoring.

Demo Model Choice

- **Baseline model for first demo**: $q25_{mix}$.
  - Existing matrix says it is the strongest complete RCA row by `recommendation_f1`.
  - RCAEval `recommendation_f1` $= 0.733$.
  - JSON format is stable in the public RCA matrix.
- **Control model**: $q25_{base}$.
  - Shows how much RCA behavior came from SFT.

Core Architecture

Memory update

$$m_t = G_\phi(m_{t-1}, c_t)$$

$$y_t = F_\theta(q_t, m_t)$$

- F_θ: frozen 7B/8B base model.
- G_ϕ: trainable wrapper.
- m_t: compact memory state.
- c_t: new context chunk or tool observation.

V0 memory type

Start with explicit memory tokens.

- Easy to train and ablate.
- Compatible with frozen decoder models.
- Safer than writing into weights.
- Hidden-state memory and LoRA memory are later ablations.

Why Freeze the 8B Base?

- We start around 8B because it is strong enough for reasoning and still feasible for repeated experiments.
- Candidate bases:
  - Llama-3.1-8B-Instruct as a secondary validation base.
  - Qwen2.5-Coder-7B or similar for public debug-trace extension.
- The base remains frozen in v0.
- Training only the wrapper limits catastrophic forgetting and makes the contribution easier to isolate.

Two Tracks

Track A: Research Paper

- New memory-wrapper architecture.
- Next-state prediction over memory states.
- Evaluate on long-context memory benchmarks.
- Main claim: compact learned memory can replace part of long prompt context.

Track B: RCA Foundation Model

- Public RCA model with internal qualitative probe support.
- Use code issues, stack traces, failing tests, patches.
- Direct prediction and agentic tool-use modes.
- Release model, report, and demo.

Dataset Plan

Track	Datasets	Purpose
Base long-context	LoCoMo, LongBench, RULER, Needle-in-a-Haystack, Qasper, NarrativeQA	Prove wrapper memory before domain-specific RCA.
Public debug traces	SWE-bench Verified, BugsInPy, Defects4J, QuixBugs, curated CodeNet/APPS traces	Later extension for debugging and trace-based root-cause reasoning.
RCA domain	Nezha, OpenRCA-500, RCAEval, lincyaw/rca	Transfer compression method to RCA evidence bundles.
Internal probe	Liang / Nokia DTS	Qualitative validation only; not for public release.

Evaluation Modes

Direct prediction

Input: issue text, code context, stack trace, logs, failing test output.
Output: root cause, evidence, fix recommendation.

Agentic RCA

The model can call tools such as search, open file, run tests, and inspect traces.
Test-time training updates the wrapper from tool observations; the frozen base does not change.

Baseline comparison

Full context vs. summary vs. retrieval vs. learned memory.

Success Metrics

Quality

- RCA / debug root-cause accuracy.
- Fix recommendation quality.
- Evidence grounding.
- JSON / schema stability for RCA outputs.

Compression

- Token and KV-cache reduction.
- Latency reduction.
- Early / middle / late evidence retention.
- Robustness to chunk order and distractors.

Budget Estimate

Scope	Time	GPUh	Cost	Output
Lean MVP	4–7 wks	1,420–2,950	\$4.3k–\$8.9k	One wrapper, smoke + RCA demo.
Paper-quality v0	7–13 wks	2,820–5,750	\$8.5k–\$17.3k	Long-context + RCA foundation model + ablations.
Strong release	10–16 wks	5,000–9,000	\$15k–\$27k	Multiple seeds, model card, polished report.

Cost model: H100 at \$3 per GPU-hour.

Execution Timeline

Phase	Duration	Decision gate
0. Smoke	3–5 days	Wrapper beats same-length summary on synthetic / NIAH.
1. Long-context	2–3 weeks	Beats summary on LoCoMo / LongBench / RULER.
2. Public debug traces	2–4 weeks	Improves direct root-cause / fix prediction on public debug traces.
3. RCA domain	1–2 weeks	Works on public RCA datasets with at least 4x compression.
4. Release polish	1–2 weeks	Ablations, demo, model card, technical report.

Current Status

- Plan 08 v0 has been scoped as a learned-memory wrapper project.
- English and Chinese notes are maintained in the research notes repo.
- Budget is separated from the full self-modifying-LLM version.
- RCA foundation-model path is defined around public RCA datasets first, with public coding/debug data as a later extension.
- Initial RCA compression scripts and report templates are drafted in the RCA demo repo.
- RCA-demo SFT initialization has identified the boundary: useful RCA learning exists, but direct SFT is not robust enough as the final RCA foundation-model route.

Next Steps

1. Build the first demo case and four prompt strategies.
2. Run frozen 8B base baselines: full context, summary, retrieval.
3. Add wrapper-memory path for the same case.
4. Package the demo table and qualitative walkthrough.
5. After demo validation, scale to LoCoMo / LongBench and public debug-trace datasets.

Ask / Discussion

- Is frozen-base + wrapper the right first milestone?
- Which 8B base should be the first public model target?
- Should the first paper emphasize long-context memory, RCA foundation modeling, or both?
- Which demo case would be most convincing for external readers?

Reporting Period: June 2026 Week 1

Date range: Mon 2026-06-01 to Sun 2026-06-07

Goal: concise progress record; detailed facts live in linked GitHub notes.

This Week: Main Message

High-level direction

Controlled model adaptation: improve RCA / long-context capability while preserving the frozen base model's general ability.

- **Plan 08 v1**: learned soft-prompt memory is now framed as a lossy-compression characterization, not a universal-memory claim.
- **Plan 08 v1.5**: next step is a do-no-harm gate that opens the wrapper only when compression is likely safe.
- **Plan 08 v2**: next version is cross-session latent memory with read/write tokens under a frozen base.
- **Related direction**: Janus tests whether long-context sites and forgetting-sensitive sites are coupled.

Problem Statement: Why This Matters

- RCA and debugging inputs are long: logs, traces, topology, prior observations, and code context often exceed cheap prompting.
- Direct SFT can teach RCA behavior, but risks weakening general instruction-following, reasoning, or retrieval behavior.
- The safer target is a **frozen-base adaptation module**: learn context compression / memory without overwriting the base model.
- This week sharpened the decision rule: use memory when it helps; close it when it would hurt.

Goal and Significance

Goal

Train a small wrapper around a frozen base model so long context can be compressed into a reusable memory signal without updating base weights.

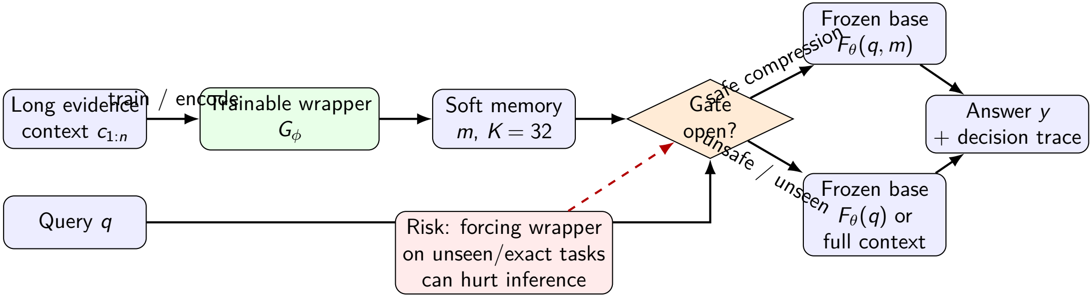
- Input: long evidence context + query.
- Output: answer from the frozen base, optionally helped by wrapper memory.

Why it matters

RCA needs adaptation to domain evidence, but direct SFT can damage general ability. The wrapper is a controlled adaptation module.

- Learn useful compression when it is safe.
- Fall back when compression would lose details.

High-Level Architecture: Inputs, Outputs, and Gate



Core idea

The wrapper is useful only when compression is safe. The gate decides whether memory should participate in inference.

Comparison: What We Are Really Testing

Approach	Benefit	Limitation
Full context	Preserves exact details	Expensive; not persistent
Direct SFT	Learns task behavior	Can regress base capabilities
Retrieval only	Good for verbatim facts	Does not learn compression / synthesis
Always-on wrapper	Learns compressed memory	Can hurt unseen or exact-retrieval cases
Gated wrapper	Uses memory only when safe	Needs reliable gate signals

Positioning

We are not claiming "wrapper replaces context." We are testing controlled adaptation: train a wrapper, measure where it helps, and gate it off when it would damage the frozen base path.

Core Evidence Chain

1. **Wrapper can train and learn something.** It improves compressible / gist-style cases relative to no-context baselines.
2. **But learned memory changes inference.** The wrapper is not neutral: on some tasks it shifts the base away from useful behavior.
3. **Unseen or exact-detail tasks are unsafe for forced memory.** If the task needs verbatim evidence or reasoning transfer the wrapper did not learn, always-on memory can lose the answer.
4. **Solution: gate between wrapper and base.** Open the wrapper when confidence is high; close it when divergence/drift says the wrapper is fighting the base.

Plan 08 v1: Main Result Cell

Benchmark	Wrapper	Gist	Best reference
QuALITY	0.193 ± 0.032	0.180 ± 0.044	no-context 0.141
MuSR-mm	0.493 ± 0.008	0.501 ± 0.013	full-context 0.551
RULER-NIAH	0.000 ± 0.000	0.000 ± 0.000	full-context 0.995

- **Conclusion**: wrapper works as a narrow lossy compressor.
- **Setting**: P08-S2 (model/data/hyperparameters).
- Helps when gist-level information is sufficient; fails when exact detail preservation is required.
- Full facts: v1 results summary.

Plan 08 v1.5: Signal Grid Cell

Signal family	Interpretation	Direction	Use in gate
Confidence / seq logprob	wrapper likely correct	positive	open
Wrapper-to-base KL / JS / TV	wrapper fights base	inverse	close
Mid/late-layer drift	late reasoning distorted	inverse	close

- **Design move**: gate on "is compression safe?" rather than forcing memory onto every input.
- **Setting**: P08-S3 (probe + signal-grid setting).
- Main signal-grid deliverables: grid folder, help ranking, non-obvious ranking.
- Compiled note: v1.5 PDF.

Plan 08 v2: Next-Version Setting

- **Question**: can latent memory be written during generation and reused across sessions?
- **Base**: still frozen; adaptation remains wrapper-only.
- **New API**: add ‘update_from_generation‘, ‘serialize‘, and ‘load‘ to the v1 wrapper protocol.
- **Differentiation**: cross-session persistence + frozen base + v1 bit-capacity diagnostic.
- **Setting**: P08-S4 (design setting; no verified v2 result yet).
- Links: v2 plan, v2 related work.

Deliverables: Where To Read More

Item	Link
Plan 08 hub	README / deliverables hub
v1 facts	main results summary (setting P08-S2)
v1.5 report	compiled PDF (setting P08-S3)
v1.5 signal grid	CSV + figures (setting P08-S3)
v2 setting	v2 plan (setting P08-S4)
Idea tables	ideas index / plans index

Next Steps

1. Implement the first do-no-harm gated wrapper using confidence and divergence features.
2. Evaluate whether gating preserves gains on compressible inputs while closing on unsafe exact-retrieval / reasoning-transfer cases.
3. Keep v1 as the characterization paper line; keep v2 as the cross-session latent-memory next version.
4. Continue Janus causal testing in parallel: protect long-context-coupled sites during SFT and measure forgetting / long-context retention.